Reproducing and Extending A Novel Hybrid Approach For Efficiently Forecasting Air Quality Data

Kumar Utkarsh P Sai Deva Charan M Charan Sree Teja D Lalit Santosh

Abstract

Air quality can change within hours. It also follows slower daily and seasonal patterns. This project studies WaveCatBoost. The method splits a pollutant series into wavelet components. It predicts each part with CatBoost and combines the forecasts. We use an R implementation and the paper's CPCB and ID1 data. We also test other learners and filters. Further studies examine cleaned CPCB data and Shillong stations. All result tables use saved project experiments. They are not copied from the paper. The Shillong results suggest that other pollutants can provide useful information. Decomposition alone does not always help. Our main lesson is simple. Data quality, fair tests, and strong baselines matter as much as model choice.

1. Introduction

Air-quality forecasting asks a simple question. Given the readings available now, what will the concentration be later? The answer is hard to predict. Pollution has short spikes and slower patterns. Some readings are missing or faulty. A smooth-looking series may still contain bad data.

The assigned paper, Borah et al. (2025), proposes WaveCatBoost. It splits the series into fast and slow parts. A CatBoost model predicts each part. The forecasts are then combined. The paper also adds a prediction band around the point forecast.

We study three questions. Does decomposition help ordinary boosting? Does cleaning the data change the answer? Can shared models or other pollutants help? We use the saved CPCB and ID1 runs. We also study cleaned CPCB data and extra Shillong results. We check the test design as well as the error scores.

We reproduce the method, not the exact published tables. The original saved benchmark uses recursive forecasts from one origin. Another workflow uses rolling one-step forecasts. These are different tasks. We keep their results separate.

2. Background and Related Work

2.1. Wavelets and component forecasting

Wavelets describe changes at different time scales. Fine components show fast changes. Coarse components show slower movement. MODWT keeps a coefficient at each timestamp (Percival & Walden, 2000). This makes it easy to align lag inputs. The components are not automatically independent. Fast changes are not always noise. Boundary handling also matters. A transform of the full series can use values that were not yet known.

2.2. Boosted trees and simple references

CatBoost combines many small decision trees. Ordered boosting reduces one type of training bias (Prokhorenkova et al., 2018). It does not remove the need for time-based validation. XGBoost (Chen & Guestrin, 2016) and LightGBM (Ke et al., 2017) are alternative tree learners. We also use simpler references. These include seasonal naive, SARIMA, persistence, and historical averages.

2.3. Accuracy and uncertainty

MAE measures the average size of an error. RMSE gives large errors more weight. MASE divides MAE by a training-period naive error (Hyndman & Koehler, 2006). It helps compare series with different scales. It needs care when training values barely change.

Conformal prediction uses past forecast errors to build prediction intervals. Standard finite-sample coverage arguments require assumptions such as exchangeability of calibration and future scores (Angelopoulos & Bates, 2023). Air-quality time series are dependent and can change with the season. A nominal 95% interval therefore needs a separate coverage check; the label alone is not evidence that it works.

3. Approach

3.1. Problem definition and input windows

Let y_t be an hourly concentration. Let X_t contain data known by hour t. A direct model predicts y_{t+h} with a

model for that horizon. A recursive model predicts one hour at a time. It adds each forecast to its history before predicting again. The original saved benchmark is recursive. Cleaned CPCB uses a direct 24-hour forecast. Its 24 inputs run from t back to $t-23$. Future targets are not inputs.

3.2. Wavelet decomposition and reconstruction

For an additive multiresolution representation,

$$y_t = \sum_{j=1}^{J} D_{j,t} + S_{J,t}. \tag{1}$$

The detail series D_j show changes at different scales. The smooth series S_J holds the remaining slow part. We use at most five levels. This gives five detail models and one smooth model. The cap limits fitting time. It is a practical choice, not a proven best level.

Each component becomes a table of past values and later targets. A regression model learns from this table. During a recursive forecast, earlier predictions replace missing future observations. We add the predicted MRA components to recover the pollutant level. Raw MODWT coefficients need their reconstruction step first.

The older periodic transform excludes the test values. Some training components can still use later training readings. The boundary can also wrap around the series. A train-only transform is therefore not always causal. A causal feature uses only values known at its own time.

3.3. CatBoost fitting and alternatives

CatBoost uses 24 component lags to predict the next value. The selected setup uses ordered boosting and depth six. A preceding validation period controls early stopping. The runner caps fitting at 120 rounds. LightGBM and XGBoost use similar lag inputs. We compare nine methods in total. These include three plain boosting models, seasonal naive, SARIMA, and four wavelet variants. We did not train new neural-network baselines. We do not mix their published scores with our own runs.

3.4. Prediction intervals

The paper's conformal interval builds a band around the WaveCatBoost point forecast. Absolute forecast error is first scaled by $\widehat{U}_t$; a trailing-window quantile of those scores then sets the half-width at a new origin:

$$CS_t = \frac{|Y_t - \widehat{Y}_t|}{\widehat{U}_t}, \tag{2}$$

$$CQ_t = \inf\Big\{q : \tfrac{1}{\min(\kappa,t-1)} \textstyle\sum_{t'=1}^{t-1} CS_{t'}\, w_{t'} \geq 1-\alpha\Big\}, \tag{3}$$

$$w_{t'} = \mathbf{1}(t' \geq t-\kappa), \tag{4}$$

$$\widehat{U}_t = \tfrac{1}{u}\textstyle\sum_{j=0}^{u-1} |e_{t-j}|, \quad e_s = Y_s - \widehat{Y}_s, \quad u = 24, \tag{5}$$

$$PI_t = \big[\widehat{Y}_t \pm CQ_t\, \widehat{U}_t\big]. \tag{6}$$

Here CS_t is the conformal score (nonconformity), CQ_t is the $(1-\alpha)$-quantile of the last κ scores, $w_{t'}$ restricts that average to the trailing window, $\widehat{U}_t$ is the rolling mean absolute residual, and PI_t is the resulting interval. We use $\alpha = 0.05$ (95% nominal coverage) and choose $\kappa \in \{24, 168, 336\}$ hours on calibration. The quantile CQ_t must be estimated before seeing the test targets.

4. Extensions Beyond the Assigned Paper

4.1. Alternative learners and filters

We test LightGBM and XGBoost as alternative learners. We also compare Haar, D4, and LA8 filters. This separates the learner choice from the filter choice. These scores come from separate saved runs. Fourier and empirical mode decomposition trials cover fewer cases. They are not part of the main nine-model ranking.

4.2. Cleaning before forecasting

The cleaned CPCB store has 8,880 hours. It excludes the final cloned CSV row. Five repeated six-pollutant patterns flag 2,637 likely artificial hours. That is 29.7% of the record. The test block contains 821 of them. These flags come from data patterns. They do not confirm the cause of each faulty reading.

We learn the repeated-pattern rules from training data. We also check bounds, stuck readings, and the $PM_{2.5}$–PM_{10} rule. Invalid readings become missing values. We do not clip them to a bound. Past-only fills supply model inputs. Invalid or filled targets are not scored. Decomposition uses a trailing 2,048-hour window. We keep only its endpoint. Each stored component therefore uses data available at that time.

4.3. Separate, shared, and multivariate models

B0 fits six separate component models. Each uses its own past values. A1 predicts the raw level. It uses target lags and other pollutants at the origin. B1 shares one learner across components. B2 adds other-pollutant components to separate models. B3 combines sharing with those extra inputs. Shared models use a component label and training-

based scaling. Standard-deviation weights keep errors in the original units. Validation selects the covariate set for B2 and B3.

The unmasked control trains on the original series. It uses the same test timestamps and valid truth as the cleaned run. This lets us compare cleaning on the same test cases.

4.4. Additional Shillong studies

The saved Shillong study covers Lumpyngngad and JN Stadium. It compares raw, multivariate, decomposed, shared, and covariate-based models. The horizons are 24, 168, 336, and 744 hours. Its model labels are local to that study. For example, Shillong A1 means target lags plus covariates. The training code was restored from the lalit branch. Its runner is `code/experiments/run_shillong_arch.R`. We have not repeated the full fits in this update. The separate short-horizon runner uses a different test design. The restored second stage adds PCA and direct, recursive, and hybrid forecasts.

5. Assumptions and Analytical Observations

We do not claim a new forecasting theorem. Two simple facts explain the design. First, assume Equation 1 holds at the target hour. The level forecast error is then the sum of the component errors. If e_j is one component error,

$$|e_{\text{level}}| \leq \sum_{j=1}^{J+1} |e_j|. \tag{7}$$

This follows from the triangle inequality. It is an upper bound. It does not prove that decomposition improves forecasts. Component errors can add up or cancel. A better detail-component score may still give a worse level forecast.

Second, each feature must use data known at its forecast origin. This rule also applies to fitted preprocessing settings. A backward-looking filter can meet it. A two-sided filter of the full series may not. We must check fills, scaling, and feature selection too. Ordered boosting cannot repair future information already present in an input.

For conformal intervals, any claim of finite-sample nominal coverage also needs the calibration assumptions of the chosen procedure. The observed seasonal shifts and flat calibration periods in this project make those assumptions questionable. We therefore report empirical failures and avoid interpreting the intervals as guaranteed bounds for future air quality.

6. Experimental Setup

6.1. Data and evidence sources

The public CPCB and ID1 files have 8,881 and 9,505 hourly rows (Borah, 2025). Each has six pollutants. Cleaned CPCB has 8,880 hours. It runs from 1 July 2022 to 5 July 2023. We keep the recorded units. CO units are uncertain, so we avoid physical-unit comparisons across pollutants.

All reported scores come from project outputs (ATS Project Group 8, 2026). We filter `results/paper/scores.csv` to CPCB and ID1. This leaves 432 rows. They cover nine methods, two sensors, six pollutants, and four windows. We exclude the extra SHILLONG group.

Cleaned predictions and comparisons are in `results/preprocessed_experiment/`. Saved Shillong evidence is in `results/shillong/architecture/`. We checked these files. We did not train fresh models for this report.

6.2. Original-data forecasting protocol

The saved original-data runs predict the final 24, 168, 336, or 744 hours. Each path starts at one origin. The windows overlap. They are not independent repeat tests. A longer window can have lower average error if it includes easier hours.

Settings chosen on the CPCB one-day split overlap the longer test windows. Periodic boundaries and artificial fills add further limits. We treat the old ranking as a recorded comparison. It is not a clean test of overall superiority.

6.3. Cleaned direct 24-hour protocol

Training targets end on 28 February 2023 at 23:00. Validation runs from 1 March to 30 April. Test targets run from 1 May to 5 July. We split by the target time, not just the origin. This keeps later targets out of earlier blocks. All five designs use the same eligible rows. There are 2,496 training rows, 757 validation targets, and 658 test targets. Final refitting uses 3,367 eligible training-plus-validation rows.

LightGBM uses L1 loss and seed 7. It runs in deterministic mode on one thread. The feature fraction is 0.8. Fitting allows up to 3,000 rounds. Early stopping waits for 100 rounds without improvement. The grid has 18 settings. Leaves are in $\{15, 31, 63\}$. Learning rates are in $\{0.03, 0.1\}$. Minimum leaf sizes are in $\{20, 50, 100\}$. Validation chooses settings and round counts. We freeze them before the final refit and test.

The MASE scale uses valid training pairs 24 hours apart. Its value is 4.356. Scaling is fitted on training data. It is updated on training plus validation for the final refit. B0 and B2 tune by component MAE. B1 and B3 use the recon-

Table 1. Measured original-data results, CPCB and ID1 only. MASE is summarised across 48 cases per method. Time is the median saved elapsed seconds per case; it is indicative rather than a controlled hardware benchmark.

Method	Mean	Median	Time (s)
WaveCatBoost	1.103	0.948	95.83
CatBoost	1.111	0.921	15.34
LightGBM	1.174	0.961	1.12
SARIMA	1.182	1.008	19.56
XGBoost	1.199	1.035	1.51
WaveSARIMA	1.223	1.026	5.08
WaveXGB	1.238	1.076	9.38
WaveLGB	1.240	0.970	7.64
Seasonal naive	1.262	1.105	0.02

structed pollutant MAE. Paired differences use a moving-block bootstrap. The block is 24 hours, with 2,000 repeats. These intervals describe this test sample. They do not cover every station or season.

6.4. Shillong protocol and limits

The Shillong study starts with 15-minute data. An hourly mean needs at least three valid readings out of four. Lumpyngngad trains on January–July 2023. It validates on August–September and tests on October–December. JN Stadium starts reporting in late July. It trains through September, validates in October, and tests in November–December. The final end-labelled hour is 1 January 2024.

The study fits a direct model for each horizon. All designs share the eligible target times. LightGBM uses learning rate 0.05. Leaves are in $\{15, 31\}$ and minimum leaf sizes are in $\{20, 100\}$. It allows 2,000 rounds and waits 100 rounds for early stopping. Predictions average seeds 2026–2028. Table 4 uses these first-stage LightGBM results. A later stage tested more learners on selected designs.

Some choices also used test-period information. Pollutant availability was checked across all three blocks. Lumpyngngad ozone was dropped after its test-period shift was seen. The resulting study is exploratory. A fresh, untouched period is needed to confirm its model choices.

7. Results and Discussion

7.1. Original-data model comparison

Table 1 covers 48 cases per method. WaveCatBoost has the lowest mean MASE, 1.103. Plain CatBoost is close at 1.111. Plain CatBoost has the better median: 0.921 against 0.948. The mean gap is about 0.008, or 0.7%. This small gap does not establish an overall winner. The overlapping windows and selection limits still matter.

WaveCatBoost takes longer to fit. Its median saved time is 95.83 seconds. CatBoost takes 15.34 seconds. WaveLGB takes 7.64 seconds, but has a higher mean MASE of 1.240. Speed and accuracy both matter. These runs do not show a universal wavelet gain.

Table 2 shows why the average is not enough. At CPCB one day, WaveCatBoost scores 0.712 and CatBoost scores 0.821. At 31 days, WaveLGB scores 0.572 and LightGBM scores 2.279. Most CPCB methods are near 1.50 over the seven-day window. At ID1 31 days, seasonal naive scores 0.548. WaveCatBoost scores 0.551. Their performance is nearly equal in that case.

7.2. Filter choice

The separate filter run compares Haar, D4, and LA8. CPCB $PM_{2.5}$ MASE is 0.727, 0.909, and 0.955, respectively. ID1 values are 0.421, 0.457, and 0.502. Haar works well in these cases. It does not win everywhere. D4 is slightly better for CPCB NO_2. LA8 is better for CPCB SO_2. Choose a filter using earlier validation data. Figures 1 and 2 show the saved comparisons.

7.3. Cleaned CPCB architecture comparison

Table 3 uses the same 658 valid test targets. Among the learned models, B0 has the lowest MAE at 7.480. B2 has the lowest RMSE at 9.917. These measures reward different behaviour. B2 can reduce large misses without having the smallest average absolute error.

The hour-of-day baseline has lower test MAE than every learned design. B0 has lower RMSE than that baseline. A simple average can therefore be competitive. The choice depends on the error measure and the cost of large misses.

The ranking changes after validation. B3 has validation MAE 6.972, against B0 at 7.257. B0 is better on the test block. Figure 3 shows this change. Validation was also used for tuning. Its best score is not an independent estimate of future performance.

Paired comparisons do not give a clear architecture winner. On test, A1 minus B0 has MAE difference $+0.36$. Its 95% bootstrap interval is $[-0.37, 0.98]$. B1 minus B0 is $+0.11$, with interval $[-0.21, 0.30]$. B2 minus B0 is $+0.58$, with interval $[-0.19, 1.36]$. All intervals include zero. B1 uses one model instead of six. Its similar score may make sharing useful for simplicity.

7.4. What cleaning changed

Cleaning lowers validation MAE for all five designs. It does not give a consistent test improvement. B0 improves from 7.757 to 7.480. Its difference interval includes zero. A1 worsens from 7.416 to 7.842. Its difference is about $+0.43$, with interval $[0.17, 0.66]$. Better validation scores

Table 2. Measured recursive $PM_{2.5}$ MASE on the original data. The day column gives the length of the terminal forecast path. LGB and Cat denote plain models; WaveLGB and WaveCat use decomposition. These overlapping windows are descriptive cases, not independent test replications.

Sensor	Days	Seasonal naive	LGB	Cat	WaveLGB	WaveCat
CPCB	1	2.021	0.737	0.821	1.031	0.712
CPCB	7	1.496	1.502	1.499	1.497	1.499
CPCB	14	0.734	0.749	0.775	0.735	0.754
CPCB	31	2.871	2.279	2.393	0.572	1.056
ID1	1	0.710	0.844	0.672	0.360	0.415
ID1	7	0.751	0.736	0.619	0.540	0.590
ID1	14	0.758	0.654	0.686	0.691	0.681
ID1	31	0.548	0.569	0.549	0.738	0.551

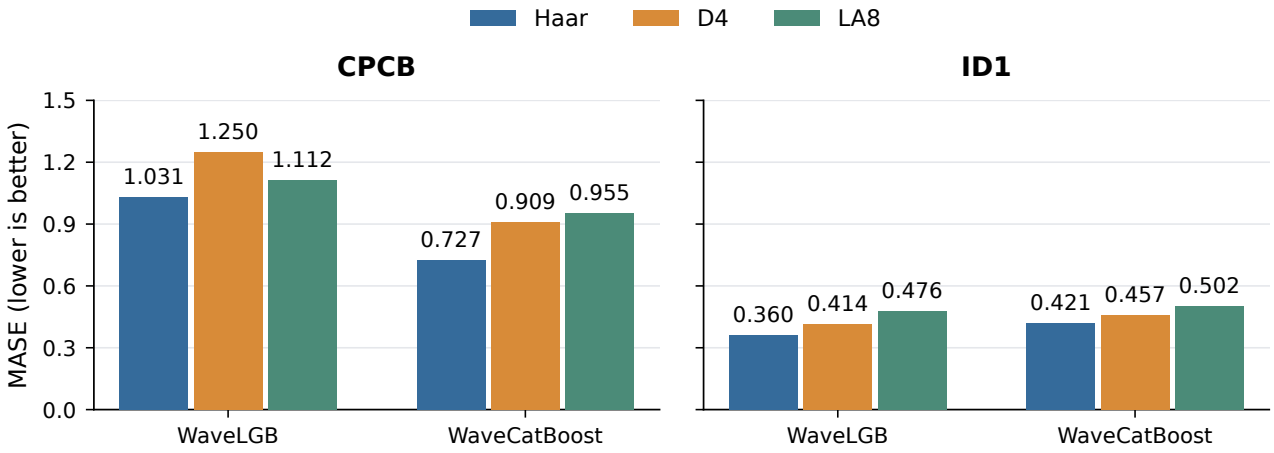


Figure 1. One-day $PM_{2.5}$ MASE for Haar, D4 and LA8. The same filter experiment compares WaveLGB and WaveCatBoost at each sensor; labels give the measured scores.

Table 3. Cleaned CPCB direct 24-hour test results. MASE uses the training scale 4.356. Baselines use the same observed targets as the learned models.

Model	MAE	RMSE	MASE
B0	7.480	10.108	1.717
A1	7.842	10.470	1.800
B1	7.591	10.292	1.743
B2	8.062	9.917	1.851
B3	7.726	9.939	1.774
Persistence	8.134	11.709	1.867
Training mean	7.382	11.250	1.695
Hour-of-day mean	7.059	10.694	1.621

did not guarantee better test scores.

Artificial fill values are still unsuitable targets. Cleaning makes the evaluation more meaningful by removing suspect truth. This is different from improving every error score. A model can score well by predicting repeated fill patterns. That need not help it predict real measurements.

All cleaned test MASE values exceed one. Their scale is the training-period seasonal-naive error. This does not mean they all lose to test persistence. B0 has test MAE 7.480, against persistence at 8.134. The test period is harder than the training scale suggests.

7.5. Retained Shillong evidence

Table 4 shows saved LightGBM results for Lumpyngngad. A2 adds other pollutant histories to the raw inputs. It improves on raw univariate A0 at all four horizons. At 168 hours, MAE falls from 15.64 to 10.94. Decomposition alone is less consistent. At 744 hours, B0 scores 15.69 and A0 scores 13.10.

JN Stadium gives less clear results. At 24 hours, A0 has MAE 23.77. A2 has 23.88 and persistence has 21.57. The record covers only a few months. Its test season differs from training. These stations do not support one universal ranking. The Lumpyngngad findings also need a fresh test after the ozone-based design change.

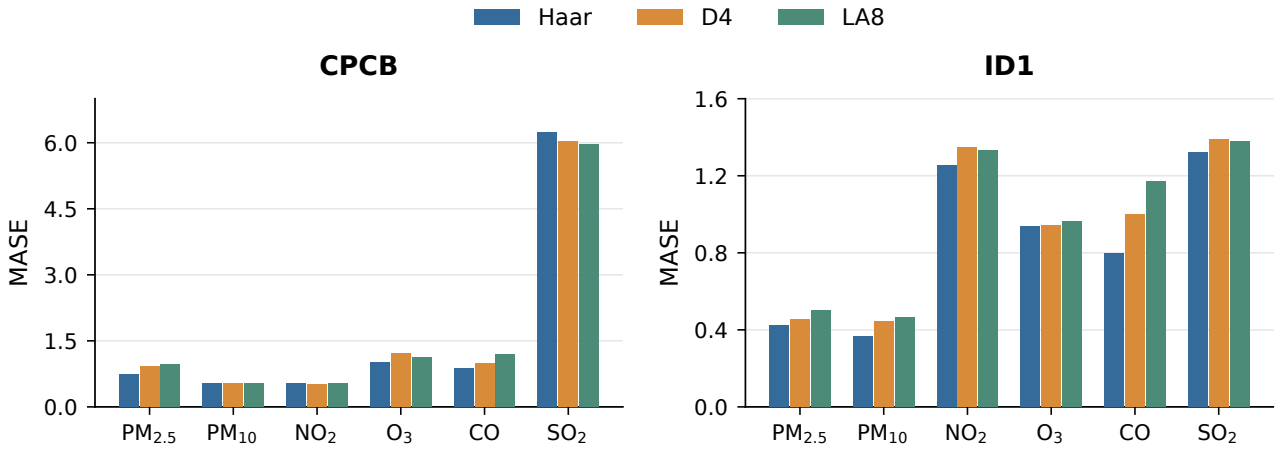


Figure 2. One-day WaveCatBoost MASE for all six pollutants and three filters. Each sensor has its own vertical scale. Haar is often competitive, but D4 and LA8 improve selected pollutant cases.

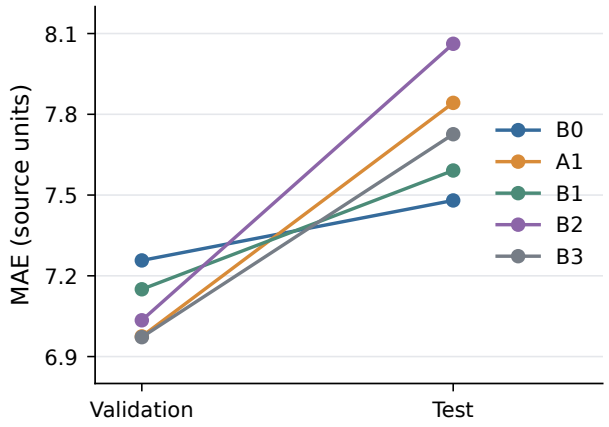


Figure 3. Cleaned CPCB MAE before and after the final temporal split. Lower is better. The architecture ranking changes between validation and test. Lines connect scores, not forecasts over time.

Table 4. Retained Lumpyngngad study: test $PM_{2.5}$ MAE. A0 is raw univariate, A2 uses multivariate lags, and B0 has separate univariate component models. Results are exploratory; see the test-inspection limits in Section 6.4.

Model	24 h	168 h	336 h	744 h
A0	9.84	15.64	16.82	13.10
A2	9.11	10.94	12.29	11.42
B0	10.38	14.21	16.09	15.69
Persistence	10.03	13.28	15.70	14.65

7.6. Uncertainty and evaluation lessons

The retained audit reports roughly 30.4% interval coverage for CPCB and 100% for ID1 in the separate seven-day calibration experiment, despite a nominal 95% target. CPCB's calibration tail is flat and produces a narrow band; ID1's band is much wider. These findings are evidence of calibration problems, not successful uncertainty quantification. Bands estimated from the test residuals are excluded from any coverage claim.

We also avoid the historical multiple-comparisons significance claim. Its saved critical-distance calculation is inconsistent with the current number of cases, and the nested horizon windows are dependent. A fresh comparison should use several forecast origins, preserve temporal dependence in its uncertainty estimates, and freeze preprocessing and model selection before the final test period.

7.7. Current recorded conformal experiment

The updated repository retains a later rolling one-step split-conformal cache in `results/paper/conformal.rds`. Table 5 and Figure 4 report this cache directly, with coverage and width recomputed from its saved bounds. This is a separate run from the older recursive-calibration audit described above. Each row covers 168 test hours after a 168-hour calibration block. Its time-varying scale uses errors observed before each successive test prediction; it is not a fixed seven-day-ahead interval. Point errors are specific to these one-step predictions and must not be pooled with the recursive scores in Table 2. Nominal 95% coverage is not attained on any of the three retained windows.

Table 5. Current recorded one-step split-conformal diagnostics for $PM_{2.5}$. Calibration and test coverage are percentages; width and MAE are in each series' source units. The target nominal coverage is 95%. Bounds are clipped at zero in the retained implementation.

Sensor	κ (h)	Cal. cover (%)	Test cover (%)	Mean width	Point MAE
CPCB	24	100.0	67.9	8.63	4.71
ID1	168	94.6	90.5	17.35	4.31
Shillong	168	94.6	90.5	127.25	50.46

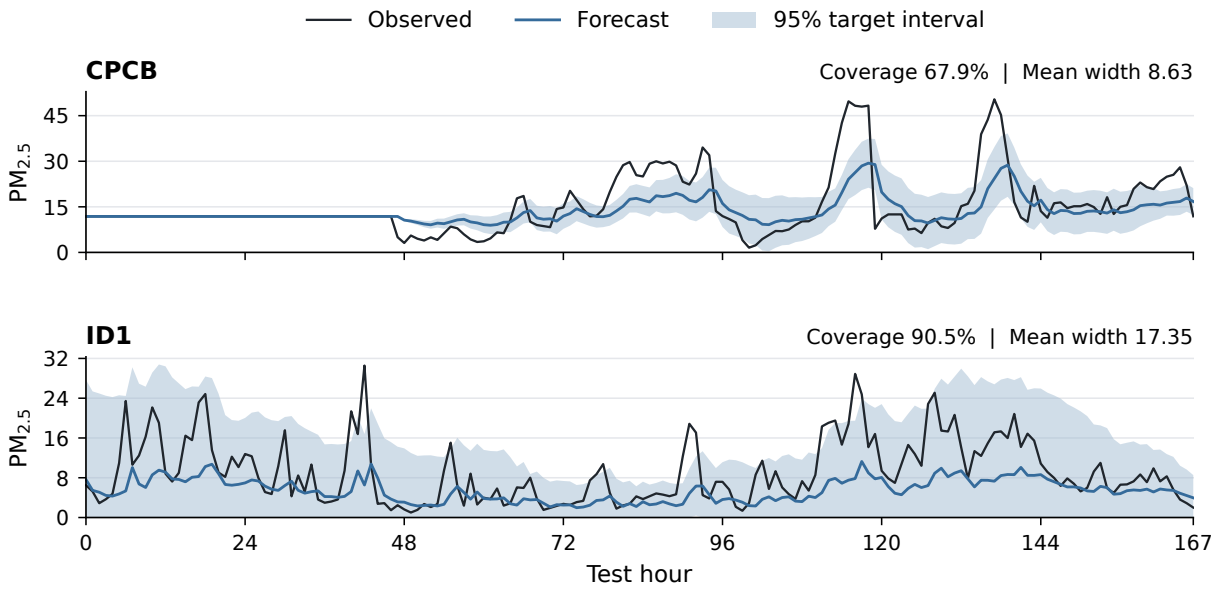


Figure 4. Current saved rolling one-step WaveCatBoost predictions and interval bounds. Each sensor uses its own vertical scale. The shaded bands are from the separate calibration cache, rather than a quantile fitted to the plotted test residuals. Coverage and widths are empirical summaries of the displayed bounds.

8. Reproducibility and Limitations

Run commands from the repository root. The main runner is `code/experiments/run_experiments.R`. The filter runner is `code/experiments/run_filter_ablation_catboost.R`. For cleaned CPCB, first run `code/preprocessing/run_preprocess.Rpaper`. Then run `code/experiments/run_preprocessed_experiment.R` with `phase1`, `phase2`, `phase3`, and `phase4`, in order. README gives the commands and expected outputs.

The YAML settings are in `code/config/`. Keep these files with the code. The shared input store is `data/processed/store/clean/`. Cleaned-CPCB intermediates use a separate store. Results remain under `results/`. To follow the method, read the loader, feature builders, decomposition, models, and metrics in that order. README maps each step to its source file.

Keep old caches before fitting again. Save the settings, seeds, package versions, and new predictions together. The current main loop also supports rolling one-step evaluation. Its fresh outputs should not be assumed to match the old recursive table. Some old summaries lack matching predictions. The larger Shillong and second-stage runners are now restored under `code/experiments/`. README gives their run order. Full fits were not repeated here. Old package versions and missing caches still limit exact numerical reproduction.

The pipeline reads the source data, builds hourly inputs, and applies the study settings. It keeps the time blocks in order. Each model uses past values to predict its target. Component models add their outputs to recover the pollutant level. Evaluation compares matching target hours. README gives the commands and output tables for each study.

9. Conclusion and Future Work

WaveCatBoost has a clear idea. Split the series, forecast each part, and combine the results. The original saved runs are competitive. Their mean gain over plain CatBoost is small. Cleaned CPCB has no clear learned-model winner. Its hour-of-day average has the lowest test MAE. Shillong

suggests that other pollutants can help more than decomposition alone.

Data and test design matter as much as the model. Artificial fills change the target. Boundary filters can use future data. Validation rankings can reverse in another season. Read MASE alongside its training scale and a test-period baseline.

Future work should use several forecast origins and years of data. Preprocessing should use only past values. A final test period should stay untouched. Raw and decomposed models should receive comparable inputs. Weather inputs must be known at the origin. Interval coverage should be checked by horizon and season.

References

Angelopoulos, A. N. and Bates, S. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. doi: 10.1561/2200000101.

ATS Project Group 8. Forecasting air quality data: R implementation, saved experiments and data audits, 2026. URL https://github.com/charan-435/Forecasting-Air-Quality-Data. Local project checkout reviewed 13 November 2026; result paths and reproducibility limits are described in this report.

Borah, J. WaveCatBoost-Time-Series-Model: Public data and implementation, 2025. URL https://github.com/jborah2/WaveCatBoost-Time-Series-Model. Companion repository of Borah et al.

Borah, J., Chakraborty, T., Nadzir, M. S. M., Cayetano, M. G., Benedetto, F., and Majumdar, S. A novel hybrid approach for efficiently forecasting air quality data. *IEEE Sensors Letters*, 9(1):1–4, 2025. doi: 10.1109/LSENS.2024.3519719. Article 6001204.

Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.

Hyndman, R. J. and Koehler, A. B. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pp. 3146–3154, 2017.

Percival, D. B. and Walden, A. T. *Wavelet Methods for Time Series Analysis*. Cambridge University Press, 2000. doi: 10.1017/CBO9780511841040.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, pp. 6638–6648, 2018.